Oplevering :

Voor de individuele oplevering maken jullie weer een implementatie van een DSO-bouwstraat. Deze keer maken jullie deze op basis van Gitea(repository manager), CI/CD (Gitea actions of Drone CI+Argo CD), inclusief monitoring met Prometheus en Graphana. In deze bouwstraat build, test en deploy je Uptime Kuma.

Tijdens de oplevering stel ik vragen over je implementatie en de keuzes die je hebt gemaakt.

Zorg dat je bouwstraat werkt tijdens je assessment zodat ik toelichting kan vragen op onderdelen.

Er komt een assessment schema op BS. Wees op tijd om (verdere) uitloop te voorkomen.

Inleveren

De volgende onderdelen moeten op BS worden ingeleverd VOOR de assessmenta. Het assessment schema komt op BS in de opdracht beschrijving.

- Een opname waar jullie jullie gerealiseerde DSO-bouwstraat demonstreren.
  Applicatie in productie -> commit change -> monitor build en test proces -> push naar productie -> aangepaste applicatie in productie. Demonstreer hier ook de werking van de optionele onderdelen of concepten.
- Configuratie bestanden. Dit zijn vaak de YAML-files.
- Beschrijving per onderdeel of dit “Onvoldoende”, “Voldoende”, “Goed” of “Zeer goed” scoort inclusief bewijs.

Beschrijving van je implementatie per onderdeel

Beschrijf in een document per onderdeel hoe dit is geïmplementeerd, inclusief bewijsstukken dat deze zijn geïmplementeerd.

Met bewijsstukken bedoel ik zaken zoals screenshots, configuraties, logfiles en ervaring.

Onderdelen van de DSO-bouwstraat

Onderdelen van de gevraagd infrastructuur:

1) Gitea als repository manager met een clone van Uptime Kuma.
2) Een CI/CD applicatie.
3) Pipeline van het builden, testen en deployen van Uptime Kuma.
4) Monitoring met Prometheus en Graphana.

DevSecOps Assessment Rubric

Assessment Criterion	Insufficient - 0 pts	Satisfactory - 6 pts	Good - 8 pts	Very Good - 10 pts
1. DevSecOps Orientation & Company/CI/CD Analysis (Assignment 1 - 25%)	Limited or incorrect understanding of DevSecOps. Sources are missing or poorly used. The company context and CI/CD requirements are unclear or unsupported.	Demonstrates a basic understanding of DevSecOps using the provided sources and at least one additional relevant source. Defines the company context and identifies appropriate DevSecOps/CI/CD requirements.	Provides a clear and well-supported analysis of DevSecOps and its industry adoption. The company context is realistic, and relevant CI/CD requirements and selection criteria are clearly identified and justified.	Demonstrates strong critical understanding of DevSecOps using high-quality sources. Critically discusses benefits, limitations, and industry challenges. Provides a thorough company-specific analysis and clearly justifies CI/CD requirements and selection criteria based on business, technical, security, and compliance needs.
2. Gitea Installation & Configuration	Gitea is not functional or cannot be demonstrated.	Gitea is correctly installed and accessible, with a working repository.	Gitea is properly configured and integrated into the development environment.	Installation and configuration are automated, reproducible, and documented, for example using Ansible or Infrastructure as Code.
3. Git Workflow, Branching & Versioning	Only basic Git usage is demonstrated; no meaningful branching or version management is used.	Appropriate use of branches and commits is demonstrated.	A clear Git workflow is used with meaningful commits, branches, merges, and release tags.	A structured release and versioning strategy is implemented and automated where appropriate, for example using semantic versioning.
4. CI/CD Pipeline Implementation	No functioning automated CI/CD pipeline is available.	A working pipeline starts automatically following an appropriate repository event.	The pipeline is clearly structured into appropriate stages and executes correctly.	The pipeline is robust, reusable, and well structured, including appropriate failure handling, conditions, artifacts, and automation.
5. Build, Test & Deployment Process	Build, testing, or deployment is missing or mainly performed manually.	The application can be built, tested, and deployed through the pipeline.	Build, test, and deployment stages are clearly separated, automated, and successfully demonstrated.	The complete process is reliable and reproducible, with appropriate validation, error handling, and controlled deployment.
6. Application Deployment & Containerization	Uptime Kuma is not successfully deployed.	Uptime Kuma is successfully deployed and accessible.	The application is correctly containerized and deployed using a reproducible configuration.	Container images are automatically built, appropriately tagged/versioned, stored in a registry, and deployed in a controlled manner.
7. Monitoring & Alerting	No functioning monitoring is available.	Basic monitoring is installed and relevant system/application information can be viewed.	Prometheus and Grafana are correctly configured with useful metrics and dashboards for relevant components.	Monitoring covers the important infrastructure and application components and includes meaningful alerting that can detect and demonstrate failures or abnormal conditions.
8. Security Controls in the CI/CD Pipeline	No meaningful security controls are implemented.	At least one relevant automated security control is integrated into the pipeline.	Multiple appropriate security controls are integrated, such as dependency scanning, vulnerability scanning, or container-image scanning, and findings are handled correctly.	Security is systematically integrated throughout the pipeline using appropriate security gates, secure secrets handling, least-privilege principles, and clearly justified controls.
9. Technical Understanding & Justification	The student cannot adequately explain the implementation or demonstrate ownership of the work.	The student can explain the main components and answer basic technical questions.	The student clearly explains technical choices, relationships between components, encountered problems, and implemented solutions.	The student demonstrates strong technical ownership, critically justifies design decisions, discusses alternatives, limitations, and security implications, and can troubleshoot during the assessment.

Bonuspunten zijn voor:

- Ansible scripting van installaties.
- Gebruik van container orchestration zoals Kubernetes en Argo CD.
- Geautomatiseerd werken met versioning, inclusief in de container releases.
- Eigen inbreng (in overleg met docent)